Soundscape

Real-Time Collaborative Music Rooms

DA219B Fullstack Lab Report

Lawrence Arciniegas

April 23, 2026

1. System Overview

Soundscape is a web application that aims to solve one problem: letting people listen to the same music at the same time and stay sync. It is designed so that you join a room and just jam to your favorite songs with a cool interphase that foucus on making it a visual experience. Although other apps like Spotify offers similar options, Soundscape is focused on the social aspect of music listening and provides an inmersive experience.

2. Database Design

Three collections with ObjectId relationships enable the core functionality:

Songs: Stores metadata (title, artist, BPM, duration, genre, coverColor). BPM field is domain-specific, ranging 40–300 for realistic music. Validation requires title and artist.

Rooms: Represents collaborative listening spaces with name, listener count, and playback state (currentSongId ref, playbackPosition, isPlaying). Name must be unique and 2–50 characters.

Queues: Maintains ordered song queues per room. Embeds songId references with addedAt timestamps and votes field (enables future democratic features). Required roomId links to parent Room.

Relationships: Room → currentSongId (One-to-One via ref), Queue → Room (Many-to-One), Queue.songs → Song (Many-to-Many array of refs).

2.1. ERD

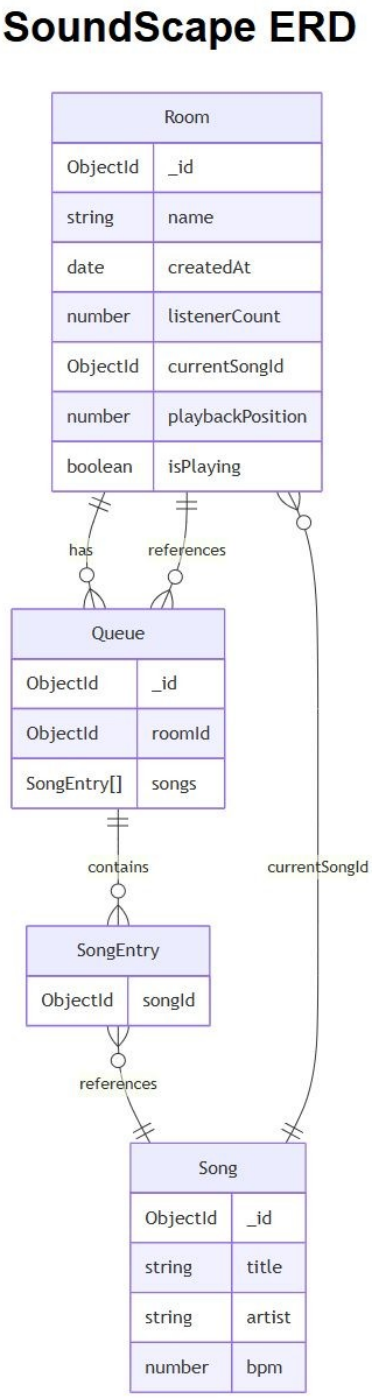

Figure 1: Entity Relationship Diagram

3. API Endpoints

3.1. Endpoint 1: POST /api/rooms/:roomId/queue

Purpose: Add a song to a room's queue with ObjectId validation.

Request: `Content-Type: application/json`

```
{
  "songId": "507f1f77bcf86cd799439012"
}
```

Response (200):

```
{
  "_id": "507f1f77bcf86cd799439020",
  "roomId": "507f1f77bcf86cd799439011",
  "songs": [
    {
      "songId": {
        "_id": "507f1f77bcf86cd799439012",
        "title": "Blinding Lights",
        "artist": "The Weeknd"
      },
      "addedAt": "2026-04-20T10:38:30Z",
      "votes": 0
    }
  ]
}
```

Error Handling: Returns 400 for missing/invalid songId, 404 if song doesn't exist, 500 on server error.

3.2. Endpoint 2: PUT /api/rooms/:id

Purpose: Update room name with duplicate prevention.

Request:

```
{
  "name": "Epic Movie Night"
}
```

Response (200): Updated room document with new name.

Error Handling: 400 if name missing, 404 if room not found, 409 if name already exists.

4. Challenges, Solutions & Learning

4.1. Challenge 1: Real-Time Synchronization

Problem: Initial approach relied on REST polling, causing visible playback desynchronization between clients and excessive database queries.

Solution: Implemented WebSocket via Socket.IO for bidirectional, event-driven communication. Clients now receive instant updates when queue changes or playback state shifts, eliminating polling overhead.

Learning: General web applications combine REST (CRUD) with WebSockets (live updates). This is a common pattern for real-time apps, and understanding when to use each is very good for performance and user experience.

4.2. Challenge 2: MongoDB Connection

Problem: `mongodb+srv://` URI failed due to DNS SRV record resolution issues. Connections to the database were refused despite valid credentials.

Solution: Switched to explicit connection string listing all three shard nodes with ports: `mongodb://host1:27017,host2:27017,host3:27017/?ssl=true&replicaSet=...`

Learning: Cloud services provide multiple connectivity strategies. While SRV offers flexibility, explicit strings in this case showed stability. Always have fallback strategies. (this took a while to figure out!)

5. Feature Iteration: Queue System Evolution

Initial (Apr 8, commit e6af04ce): Queue stored flat song ObjectId array with no metadata.

```
songs: [ObjectId, ObjectId, ObjectId]
```

Problem: No insertion timestamps, no voting capability, lost context about when/who added songs.

Improved (Apr 20, commit ee3146c6): Enhanced to embedded documents capturing metadata.

```
songs: [{
  songId: ObjectId,
  addedAt: Date,
  votes: Number
}]
```

Impact: Enables temporal ordering, this for future features (vote-to-skip).

6. Extra Libraries & Technologies

- **Socket.IO**: Real-time WebSocket communication with fallback support
- **Mongoose ODM**: Schema validation, population, atomic operations on MongoDB
- **CORS**: Cross-Origin Resource Sharing for frontend-backend communication
- **dotenv**: Environment variable management for secrets (.env)
- **concurrently**: Single-command startup for frontend and backend
- **StackOverflow**: Troubleshooting MongoDB connection issues and Socket.IO integration patterns
- **Claude**: Assistance with API design, error handling strategies, and report structuring
- **GitHub Copilot**: Code suggestions for Express routes, Mongoose schemas, and React components
- **Latex**: Report formatting and structuring for clear presentation of technical details

7. How I Would Approach Differently

1. **Start with API Design**: Define API contracts and data models first for a having a better development
2. **API-First Design**: Define API specs before implementation for better frontend-backend alignment
3. **Separate Authentication**: Implement user accounts and JWT tokens earlier to support multi-user scenarios properly (might implement in the future)
4. **Testing from Start**: Jest/Supertest for backend, Vitest for frontend. Catch bugs early
5. **Database Indexing**: Add indexes on frequently queried fields (roomId, userId) for production performance
6. **Error Logging**: Implement centralized logging (Winston/Morgan) instead of console.log for production debugging
7. **API Rate Limiting**: Add middleware to prevent abuse before security becomes an afterthought

A. Additional Resources

A.1. Git Commit Timeline

The development followed a logical progression from infrastructure setup through feature implementation to bug fixes and documentation altought not all of them followed the specific format of fix, feat, or refactor, they show the progressive evolution of the project.

Date	Commit Hash	Description
Apr 8	`10fb2cd6`	Initial commit
Apr 8	`99dc148e`	npm setup for root, frontend, backend
Apr 8	`1a190a5d`	Following commit for previously non-staged changes
Apr 8	`2884b1a1`	env and .gitignore
Apr 8	`156b5d82`	MongoDB connection (standard string, SRV troubleshoot)
Apr 8	`e6af04ce`	Routes, controllers, server.js
Apr 8	`94b6149e`	Frontend views, Socket.IO, package.json update
Apr 8	`27e5a343`	README update
Apr 20	`dec50195`	.env.example for good practice
Apr 20	`ee3146c6`	PUT endpoint, ObjectId validation, edit UI
Apr 20	`566e1fa9`	ERD diagram generated
Apr 21	`3cacae2a`	Bug fixes: roomController, queueController, audio cleanup
Apr 21	`3b3a0a2f`	Problem stated in README
Apr 21	`c0825154`	README update
Apr 21	`800a1811`	LaTeX report first version (buggy)

Table 1: Development Timeline